Confidence-Aware Graph Neural Networks for Social Bot Detection: Message-Passing Reweighting and Similarity-Graph Construction

Kassymkhan Kanalbay
Astana IT University (AITU)
Astana, Kazakhstan
255479@astanait.edu.kz

La Lira L'vovna
Candidate of Physical and Mathematical Sciences, Associate Professor
L.N. Gumilyov Eurasian National University
Astana, Kazakhstan
Lira.La@astanait.edu.kz

***Abstract*—Social bots distort engagement metrics and amplify disinformation, which motivates graph neural networks (GNNs) that exploit relations between accounts. Standard Graph Convolutional Networks (GCNs) weight neighbor messages by node degree only, so a neighbor with a confident prediction counts as much as a borderline one. We study confidence-aware message passing for bot detection: a Confidence-Aware GCN that reweights each neighbor by the model's own maximum softmax probability from the previous epoch, and three extensions, namely an adaptive variant that applies confidence on top of the baseline's own normalization with a learnable, layer-wise exponent (AC-GCN, AC-SAGE), agreement-based edge weights, and pseudo-label self-training. On a labeled Twitter dataset (5 000 accounts, 5-fold cross-validation $\times$ 5 seeds, paired Wilcoxon tests with Holm correction) we also study how the similarity graph is built. The construction of the graph is the decisive design choice: a cosine k-NN graph over profile descriptions (edge homophily 0.62) makes every GCN-type model worse than a feature-only MLP (F1 0.740 for GCN vs. 0.807), whereas a k-NN graph over account metadata (homophily 0.78) turns all GNNs into the best detectors. GraphSAGE on the metadata graph reaches F1 0.840 and AUROC 0.921, i.e. $+3.2$ F1 over the MLP and $+9.9$ F1 over the original description-graph GCN ($p < 10^{-6}$). With only 250 or 500 labeled accounts, GraphSAGE with confidence-based self-training is the best model and beats the MLP by about 2 F1 points. The adaptive confidence models remove the AUROC loss of the original Conf-GCN (0.819 vs. 0.794 on the description graph) and stay within 0.5 F1 points of their baselines, but no confidence-weighting scheme improves on them; their learned exponents shrink below one, and on real data the weights deviate from uniform aggregation by only 6.5%. The earlier single-split precision gain of Conf-GCN does not replicate. We release code, per-run results and all scripts.**

***Index Terms*—bot detection, graph neural network, message passing, confidence weighting, self-training, graph construction**

I. Introduction

Social media platforms increasingly contend with coordinated inauthentic accounts, commonly called social bots, that inflate follower counts, manufacture trending topics, and propagate disinformation at scale [6]. Bots frequently operate in coordinated clusters and use templated, repetitive language, so both the relational structure of a social graph and the semantic content of posts carry complementary signal. This has motivated a shift from per-account classifiers toward Graph Neural Networks (GNNs), which propagate information along follow, mention, reply, or content-similarity edges and can therefore detect coordinated behavior invisible to a single-account feature vector.

A standard GCN [1] aggregates each node's neighborhood with degree-normalized weights that are agnostic to how reliable a neighbor's representation currently is. In bot detection this is a potential weakness: when bots are deliberately disguised to resemble genuine users, a small number of cross-class edges connect bot clusters to the human subgraph, and an ambiguous account near the decision boundary then contributes to its neighbors' updates with the same weight as an account classified with near-certainty. Noisy, low-confidence signal can thus leak across the class boundary.

This paper studies a Confidence-Aware GCN that addresses this weakness at the message-passing level. Each neighbor's contribution is scaled by the model's own confidence in that neighbor's class prediction from the previous epoch, renormalized so incoming weights sum to one. The mechanism requires no additional learnable parameters. In a preliminary single-split experiment the method raised precision by 3.0 points at a 7.6-point recall cost. Here we test whether that trade-off is a genuine effect of the confidence signal.

Research questions. **RQ1**: does the mechanism behave correctly (convergence; equivalence to uniform aggregation when all confidences are equal)? **RQ2**: how does it change accuracy, precision, recall, F1, AUROC and AUC-PR relative to GCN, GraphSAGE, GAT and a feature-only model, and is the change statistically reliable? **RQ3**: is the effect attributable to the confidence signal itself, rather than to the change in normalization or to a shift of the decision threshold? **RQ4**: how much does the construction of the similarity graph matter compared with the aggregation rule? **RQ5**: do natural extensions of the method (learnable exponent, confidence on top of the baseline normalization, agreement weights, self-training) help? **RQ6**: what changes when only a few hundred accounts are labeled?

Contributions. (1) A parameter-free confidence-weighted message-passing layer and an adaptive family (AC-GCN, AC-SAGE) that reduces exactly to GCN or GraphSAGE at $\alpha = 0$

TABLE I
POSITIONING RELATIVE TO RELATED AGGREGATION SCHEMES

Method	Weight source	Extra params	Uses pred. confidence
GCN [1]	degree	no	no
GAT [2]	learned attention	yes	no
GraphSAGE [3]	mean/pool	yes	no
ConfGCN [12]	learned label confidence	yes	indirectly
Ours	softmax confidence	no	yes

and learns how much confidence to use, all as PyTorch Geometric layers with unit tests of their formal properties. (2) A rigorous evaluation on a real Twitter dataset with repeated cross-validation, paired significance tests, a threshold-matched comparison and ablations that separate the confidence signal from its confounders. (3) A graph-construction study showing that a metadata k-NN graph turns GNNs from worse than an MLP into the best detectors; the resulting GraphSAGE detector improves F1 by 9.9 points over the original pipeline. (4) A label-scarce study in which confidence-based self-training on the improved graph gives the best detector. (5) An honest account of what does not work: confidence reweighting alone yields no significant gain, and we explain why. Code, configurations and per-run results are public.[1]

II. Related Work

A. Feature-based bot detection

Early detectors relied on hand-engineered behavioral and metadata features (posting frequency, account age, follower-to-friend ratios) combined with supervised classifiers; Botometer (formerly BotOrNot) blends a large number of such features in a supervised ensemble [4]. Such approaches are effective against unsophisticated bots but degrade as bots adopt human-like patterns, and generalize poorly across datasets [5], [6]. Feature selection studies on Twitter bot detection show that profile and description features alone reach moderate accuracy [11].

B. Graph-based bot detection

Graph-based approaches exploit the observation that bots, even when individually convincing, tend to be structurally distinguishable: they cluster densely, share near-duplicate content, or connect to common amplification targets. GCNs [1] and inductive GraphSAGE-style models [3] have been applied to spam and bot detection [7]. BotRGCN and the Relational Graph Transformer extend this to heterogeneous graphs with multiple relation types [8], [10], evaluated on the TwiBot-20 benchmark [9]. Graph Attention Networks (GATs) learn per-edge attention weights instead of fixing them to degree-based normalization [2]. The benefit of message passing depends strongly on homophily, i.e. the fraction of edges joining same-class nodes [23].

[1]https://github.com/solodancer235-star/confidence-aware-gcn-bot-detection

C. Confidence- and uncertainty-aware aggregation

Several lines of work use model confidence. Pseudo-labeling and self-training use confident predictions on unlabeled data as additional supervision [13]. The closest graph-learning work is ConfGCN [12], which estimates a per-node confidence over label distributions and uses it to modulate neighbor influence in semi-supervised node classification. Our approach differs in the signal and its cost: ConfGCN maintains a learned, per-node label-distribution estimate with additional parameters and a distribution-matching objective, whereas we derive confidence directly from the classifier's own softmax output, with no extra parameters, and study it specifically for bot detection. The reliability of softmax confidence itself is known to be imperfect and can be improved by calibration such as temperature scaling [14]. Stochastic regularizers on message passing, such as DropEdge [15], also aim to reduce the effect of noisy neighbors, but do so randomly and without regard to prediction reliability.

D. Similarity graphs for tabular data

Sentence-BERT embeddings [16] enable similarity-based edges between accounts posting near-duplicate content even without explicit social links. For tabular bot datasets that provide rich per-account metadata but no follow graph, a k-nearest-neighbor graph over embeddings is a common substitute for explicit structure, a construction long used in graph-based semi-supervised learning [17].

III. Problem Formulation

Let $G = (V, E)$ be a social graph, where V is the set of accounts and E the set of follow, mention, reply, or content-similarity edges. Each node v has a feature vector $x_v \in \mathbb{R}^d$ built from a Sentence-BERT embedding of the account text concatenated with normalized profile metadata. A label $y_v \in \{0, 1\}$ (0 = human, 1 = bot) is known for a subset of nodes. The task is transductive semi-supervised node classification: learn $f : G \to \{0, 1\}^{|V|}$ predicting the label of every node, including those withheld from training, by propagating information along E.

IV. Methodology

A. Baseline GCN

The baseline stacks three `GCNConv` layers (PyTorch Geometric) mapping $d \to 64 \to 64 \to 2$, with ReLU and dropout ($p = 0.5$). Node i is updated at layer l as

$$h_i^{(l+1)} = \sigma\Big(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\hat{d}_i \hat{d}_j}} W^{(l)} h_j^{(l)} \Big), \qquad (1)$$

where $\hat{d}_i$ is the degree of i including a self-loop. Training uses cross-entropy with Adam [19].

TABLE II
CONFIGURATION OF BASELINE AND CONFIDENCE-AWARE MODELS

Component	Baseline	Conf.-Aware
Conv. layer	`GCNConv` (PyG)	`ConfWeightedConv`*
Layers	3	3
Hidden dim.	64	64
Dropout	0.5	0.5
Learnable params (Twitter)	29 570	29 570
Optimizer	Adam (selected on val.)	Adam (selected on val.)

*Custom `MessagePassing` layer; mean-style normalization, no self-loop.

B. Confidence-Aware GCN

The confidence of node j is the maximum softmax probability of its logits z_j from the previous epoch:

$$c_j = \max_k \ \mathrm{softmax}(z_j)_k. \tag{2}$$

For $C = 2$ classes, $c_j \in [0.5, 1]$. For every directed edge $j \to i$ the weight is renormalized over the incoming edges of i:

$$\tilde{w}_{j\to i} = \frac{c_j^{\alpha}}{\sum_{k\in\mathcal{N}(i)} c_k^{\alpha}}, \tag{3}$$

with $\alpha = 1$ in the proposed model, and the node update becomes

$$h_i^{(l+1)} = \sigma\Big(\sum_{j\in\mathcal{N}(i)} \tilde{w}_{j\to i}\, W^{(l)} h_j^{(l)} \Big). \tag{4}$$

Confidence is initialized to 1 for every node. At each epoch a forward pass produces logits for the loss and optimizer step; a second no-gradient forward pass in evaluation mode then recomputes the confidence vector for the next epoch. Confidence is stored as a plain tensor and updated inside a no-gradient context, so gradients never flow through it; this avoids a feedback loop between the loss and a quantity that depends on it. Confidence is computed from model outputs only and uses no labels, so evaluation nodes are not leaked.

C. Properties and limits of the mechanism

Reduction to uniform aggregation. When $c_j = 1$ for all j (the first forward pass), Eq. (4) is mean aggregation over $\mathcal{N}(i)$. This is not identical to the symmetric degree normalization with self-loops used by the baseline (Eq. 1). The two models therefore differ in normalization and self-loop handling in addition to the confidence signal; we address this explicitly with ablations (Section VI-E) rather than assuming the difference is attributable to confidence alone. The layer is also invariant to a global rescaling of the confidences, since only ratios enter Eq. (3).

Bounded influence. Because $c_j \in [0.5, 1]$ in the binary case, the weights of any two neighbors of the same node differ by at most a factor of 2^{α}. With $\alpha = 1$ the mechanism can therefore only moderately rebalance a neighborhood; we measure how much it actually does in Section VII-I.

D. Architectural parity

Both models use three layers, hidden dimension 64, dropout 0.5 and the same hyper-parameter search (Table II); they have exactly the same number of learnable parameters (verified by a unit test). Parity of capacity does not imply that confidence is the only difference, so we complement it with the ablations below.

E. Adaptive confidence-aware GNNs

The ablations in Section VII-D show that Conf-GCN differs from GCN mainly through its normalization. We therefore propose an adaptive variant in which confidence only *redistributes* the baseline's own weights. For AC-GCN, with $a_{j\to i} = 1/\sqrt{\hat{d}_i \hat{d}_j}$ over $\mathcal{N}(i) \cup \{i\}$ as in Eq. (1),

$$w_{j\to i}^{(l)} = a_{j\to i}\, c_j^{\alpha_l}\, \frac{\sum_k a_{k\to i}}{\sum_k a_{k\to i}\, c_k^{\alpha_l}}, \tag{5}$$

which keeps the row mass of GCN; with constant confidence or $\alpha_l = 0$ the layer is *exactly* `GCNConv`, so any difference from GCN is due to confidence alone. The exponent $\alpha_l = \mathrm{softplus}(\rho_l) \geq 0$ is learned per layer (three scalars, initialized at $\alpha = 1$); large α removes the factor-2 bound on neighbor influence. AC-SAGE applies the same idea to GraphSAGE: $h_i' = W_r h_i + W_n \sum_{j\in\mathcal{N}(i)} \tilde{w}_{j\to i} h_j$ with $\tilde{w}$ from Eq. (3) and learnable α_l; at $\alpha = 0$ it equals mean-aggregation GraphSAGE.

Agreement weights. Node confidence ignores *whether* the two endpoints of an edge agree. We therefore also test edge factors $f_{j\to i} = (\sum_k p_{jk} p_{ik})^{\alpha_l}$, the probability under the previous epoch's predictions that i and j share a class, inserted in Eq. (5) instead of $c_j^{\alpha_l}$. Confidently disagreeing endpoints receive a weight close to zero, so the edge is effectively pruned.

Self-training. Confidence can also be used as supervision [13], [24]. After 20 warm-up epochs, every node outside the training set whose previous-epoch confidence is at least $\tau = 0.9$ receives its predicted class as a pseudo-label, added to the loss with weight $\lambda = 0.5$. True labels of these nodes are never used. Self-training is applied to GCN, GraphSAGE, AC-GCN and AC-SAGE ("+ST").

V. IMPLEMENTATION

The system is implemented in Python with PyTorch and PyTorch Geometric [20], sentence-transformers (`all-MiniLM-L6-v2`, 384-dimensional output) [16], NetworkX for graph construction, scikit-learn for k-NN graphs, scaling and metrics, and SciPy for statistical tests.

`ConfidenceWeightedConv` extends `MessagePassing` with additive aggregation. Its `forward` method takes a per-node source-weight vector as an argument, computes per-edge weights as the source node's weight divided by the sum of weights over the target node's incoming edges (scatter-add indexed by target), and its `message` method multiplies each transformed source representation by this weight. `ConfidenceAwareGCN`

stacks three such layers, stores confidence as a plain module attribute (not a registered parameter), lazily initializes it to ones on the first forward call, and exposes a no-gradient `update_confidence` method that recomputes it as the row-wise maximum softmax probability of the latest logits. `ConfidenceGCNConv` and `ConfidenceSAGEConv` implement Eq. (5) and AC-SAGE; their source weights are not detached, so the exponents receive gradients. A `mode` switch implements the ablation controls (uniform, shuffled and inverted confidence) and a `self_loop` flag adds self-loops inside the confidence-weighted layer. Unit tests check that incoming weights sum to one, that $c \equiv 1$ reproduces mean aggregation exactly, scale invariance, parameter parity with `GCNConv`, that confidence carries no gradient, that `ConfidenceGCNConv` equals `GCNConv` for constant confidence and preserves row mass, and that the exponents receive finite gradients.

VI. Experimental Setup

A. Synthetic social graph

To verify correctness under controlled conditions we generate a graph of 500 nodes: 400 humans connected by a Barabási–Albert preferential-attachment model ($m = 3$), and 100 bots in two dense, fully connected clusters of 50, plus 20 random bot–human edges simulating disguised bots (3 661 structural edges). Human text is drawn from 15 topical templates, bot text from 10 near-duplicate cryptocurrency-spam templates with minor lexical variation. Each node's 384-dimensional embedding is its feature vector, and edges are added between node pairs whose cosine similarity exceeds 0.85 (6 663 new undirected edges, 10 324 in total).

B. Real-world Twitter dataset

We use the publicly available Twitter human/bot dataset [21] (37 438 accounts, 25 013 humans, 12 425 bots), with labels obtained through the Botometer API combined with manual verification and 20 metadata columns. We work with a cleaned public redistribution of it [22] that keeps the 29 407 accounts whose interface language is English or unset (19 676 humans, 9 731 bots; class ratio 2.02:1 vs. 2.01:1 in the original) and strips URLs, @-mentions, e-mail addresses and emoji from the descriptions. Because labels partly derive from a feature-based detector, label noise correlated with metadata features is possible (Section VIII-E).

For computational reasons on CPU, we draw a stratified subsample of 5 000 accounts (2 500 humans, 2 500 bots; fixed seed); 1 516 of them (30%) have an empty description. Node features are 394-dimensional: a 384-d embedding of the profile description, six numeric fields (status, follower, friend and favorite counts, account age in days, average tweets per day) log-transformed and standardized with training-node statistics of each fold, and four binary fields (default profile, default profile image, verified, geo-enabled).

Graph construction. As the dataset has no explicit follow graph, we build three label-free, symmetrized cosine k-NN graphs ($k = 10$) [17]: the *description graph* over description embeddings (used in the preliminary experiment; 84 462 directed edges, edge homophily 0.617); the *metadata graph* over the ten z-scored metadata fields (66 440 edges, homophily 0.777); and the *feature graph* over the description embedding concatenated with the metadata block scaled by $1/\sqrt{10}$ so that both blocks have comparable norm (77 696 edges, homophily 0.756).

C. Compared models

All GNNs use three layers, hidden dimension 64 and dropout 0.5:

- **MLP**: same features, no graph (isolates the value of structure).
- **GCN** [1]: baseline.
- **GraphSAGE** [3]: mean aggregator with a separate root (self) transformation.
- **GAT** [2]: learned attention; total hidden width 64 split over h heads.
- **Conf-GCN** (ours, Section IV-B).
- **AC-GCN, AC-SAGE, agreement-weighted GCN** and **+ST** variants (ours, Section IV-E); they reuse the hyper-parameters selected for GCN or GraphSAGE.

D. Evaluation protocol

Splits and repetition. Each model is evaluated with stratified 5-fold cross-validation repeated over 5 random seeds (25 runs per model), with the training portion of each fold further split to obtain a validation set (60/20/20 overall). All models see identical splits.

Hyper-parameter selection. Learning rate $\in \{0.01, 0.005\}$, weight decay $\in \{5\times10^{-4}, 5\times10^{-5}\}$ and, for GAT, the number of heads $\in \{1, 4, 8\}$ are selected by mean validation F1 over the five folds of seed 0 only; the test folds are never used. Selected: MLP, GraphSAGE and GAT ($h = 1$) $\mathrm{lr} = 0.005$, $\mathrm{wd} = 5 \times 10^{-4}$; GCN and Conf-GCN $\mathrm{lr} = 0.01$, $\mathrm{wd} = 5 \times 10^{-5}$. All ablations reuse the Conf-GCN setting. The epoch count is fixed at 100 for all models, without early stopping.

Metrics. Accuracy, precision, recall and F1 for the bot class at the default threshold 0.5, plus the threshold-independent AUROC and AUC-PR (average precision), and the expected calibration error (ECE, 10 bins).

Statistical testing. We report mean $\pm$ standard deviation over the 25 runs and compare Conf-GCN against each baseline with a paired two-sided Wilcoxon signed-rank test over matched runs, with Holm correction across the comparisons of a table [18], [25]. Because the runs share data across folds and seeds, p-values are treated as indicative rather than exact.

Label-scarce regime. To mimic moderation settings with few verified labels, we fix 20% test and 20% validation nodes and train on a stratified set of only 100, 250 or 500 labeled accounts (2, 5, 10% of the graph), repeated over 10 random splits, on the feature graph.

Threshold-matched analysis. A precision gain with a recall loss could simply reflect a different operating point on the same ranking. For every run we therefore (i) take the validation

TABLE III
SYNTHETIC GRAPH, SINGLE STRATIFIED 60/20/20 SPLIT

Metric	Baseline	Conf.-Aware	Δ
Accuracy	1.0000	1.0000	0.0000
Precision	1.0000	1.0000	0.0000
Recall	1.0000	1.0000	0.0000
F1-score	1.0000	1.0000	0.0000

recall of Conf-GCN at threshold 0.5 as a target, (ii) choose each baseline's threshold on *its validation set* as the largest threshold reaching that recall, and (iii) compare test precision at these thresholds. We also report full precision–recall curves.

E. Ablations

To attribute the effect to the confidence signal itself, we run, with all other settings fixed:

1) **Uniform-confidence control**: same layer with $c_j \equiv 1$ (mean aggregation, no self-loop). This separates the effect of confidence from the change in normalization.
2) **Shuffled confidence**: confidences randomly permuted across nodes each epoch (same marginal distribution, no node-specific information).
3) **Inverted weighting**: weights proportional to $1-c_j+0.5$, favoring low-confidence neighbors.
4) **Exponent** α: $\alpha \in \{0.5, 2, 4\}$ ($\alpha = 0$ is the uniform control, $\alpha = 1$ the full model).
5) **Self-loop**: self-loops added to the confidence-weighted layer, with real and with uniform confidence.
6) **Graph density**: $k \in \{5, 10, 20\}$ in the k-NN graph, for GCN and Conf-GCN.
7) **Graph construction**: description, metadata and feature graphs for all GNNs.

VII. RESULTS

A. Synthetic graph (RQ1)

Both models reach the task ceiling: accuracy, precision, recall and F1 are all 1.0000 for the baseline and the confidence-aware model (Table III, Fig. 1); the same holds for MLP, GraphSAGE and GAT and for all 25 cross-validation runs of every model (125 runs, every metric 1.000). This is expected from the data-generating process: the bot clusters are fully connected subgraphs whose members share near-identical embeddings, producing a dense web of semantic edges within each cluster and almost none to the human subgraph, so the classes are separable by structure and by features alone. The task therefore verifies correctness (RQ1) but cannot discriminate between the models. Training loss falls to below 10^{-3} (9.8×10^{-4} for GCN, 6.5×10^{-4} for Conf-GCN) for both models, and the mean confidence of the confidence-aware model rises from 0.998 at epoch 20 to 0.999 at epoch 100 (Fig. 4), showing that the mechanism converges as intended. The unit tests confirm the formal properties of Section IV-C, in particular exact equivalence to mean aggregation when $c \equiv 1$.

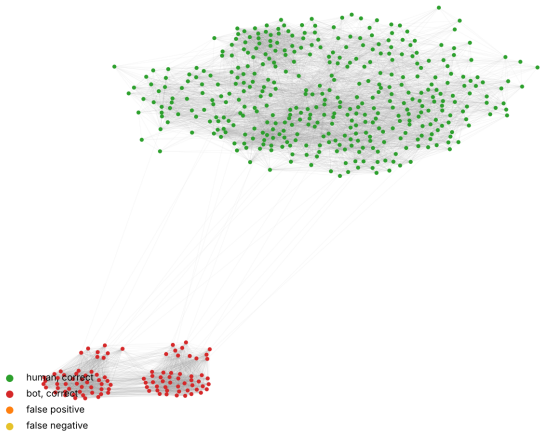


Fig. 1. Synthetic graph (500 nodes, 10 324 edges) colored by the Conf-GCN prediction: correctly classified humans (green) form a loose preferential-attachment community; correctly classified bots (red) form two dense, well-separated clusters. No node is misclassified.

TABLE IV
REAL TWITTER DATA, SINGLE SPLIT, PRELIMINARY EXPERIMENT

Metric	Baseline	Conf.-Aware	Δ
Accuracy	0.7150	0.7130	−0.0020
Precision	0.7002	0.7300	+0.0298
Recall	0.7520	0.6760	−0.0760
F1-score	0.7252	0.7020	−0.0232

B. Preliminary single-split result

Table IV reports the preliminary single train/test split on a 5 000-account subsample of the full 37 438-account release. The confidence-aware model improved precision by 2.98 points while recall fell by 7.60 points and F1 by 2.32 points; accuracy was essentially unchanged. On a single split, differences of this size cannot be separated from sampling noise, which is why the protocol of Section VI-D was added. Absolute numbers below are not directly comparable with Table IV because the English-only redistribution yields a different subsample.

C. Extended comparison (RQ2)

Table V shows the cross-validated comparison. Three findings stand out. First, the precision gain of Table IV does not replicate: Conf-GCN and GCN have the same mean precision (0.732 vs. 0.731, $p = 0.83$). The recall loss does replicate in direction (-2.0 points), so Conf-GCN has a lower F1 than GCN (0.731 vs. 0.740, $p = 0.004$) and is statistically indistinguishable from GAT in F1. Second, the threshold-independent metrics are clearly worse for Conf-GCN: AUROC 0.794 vs. 0.819 for GCN and GAT, AUC-PR 0.772 vs. 0.824, i.e. its ranking of accounts is worse, not merely shifted. Third, and most importantly for interpreting any GNN result on this benchmark, the feature-only MLP is the best model (F1 0.807, AUROC 0.890), tied with GraphSAGE (F1 0.803), whose

TABLE V

REAL TWITTER DATA (DESCRIPTION k-NN GRAPH, $k = 10$), 5-FOLD CV $\times$ 5 SEEDS (MEAN $\pm$ STD OVER 25 RUNS). p: PAIRED WILCOXON VS. CONF-GCN, HOLM-CORRECTED OVER THE FOUR BASELINES.

Model	Acc.	Prec.	Rec.	F1	AUROC	AUC-PR	p (F1)	p (Prec.)
MLP (no graph)	0.807±0.008	0.805±0.012	0.809±0.023	0.807±0.010	0.890±0.008	0.894±0.009	<0.001	<0.001
GCN	0.737±0.014	0.731±0.020	0.752±0.037	0.740±0.016	0.819±0.013	0.824±0.014	0.004	0.833
GraphSAGE	0.803±0.011	0.805±0.014	0.801±0.030	0.803±0.014	0.887±0.010	0.893±0.009	<0.001	<0.001
GAT	0.737±0.014	0.744±0.021	0.726±0.045	0.734±0.020	0.819±0.013	0.823±0.013	0.230	0.003
Conf-GCN (ours)	0.731±0.016	0.732±0.023	0.732±0.042	0.731±0.019	0.794±0.013	0.772±0.013	—	—

TABLE VI

ABLATIONS, DESCRIPTION GRAPH (MEAN $\pm$ STD OVER 25 RUNS). p: PAIRED WILCOXON OF F1 VS. FULL CONF-GCN, HOLM-CORRECTED OVER THE NINE COMPARISONS.

Variant	Prec.	Rec.	F1	AUROC	p (F1)
GCN (reference)	0.731±0.020	0.752±0.037	0.740±0.016	0.819±0.013	0.015
Uniform confidence ($c \equiv 1$)	0.730±0.021	0.745±0.039	0.736±0.018	0.795±0.013	0.026
Shuffled confidence	0.729±0.024	0.741±0.033	0.735±0.017	0.795±0.013	0.104
Inverted weighting	0.736±0.017	0.731±0.036	0.733±0.020	0.797±0.013	0.668
$\alpha = 0.5$	0.730±0.025	0.739±0.039	0.734±0.016	0.795±0.013	0.668
$\alpha = 2$	0.733±0.028	0.725±0.042	0.728±0.017	0.793±0.014	0.668
$\alpha = 4$	0.725±0.026	0.733±0.044	0.727±0.017	0.788±0.014	0.668
Uniform conf. + self-loop	0.729±0.023	0.756±0.034	0.742±0.016	0.817±0.013	<0.001
Conf-GCN + self-loop	0.730±0.028	0.741±0.051	0.734±0.018	0.815±0.013	0.668
Conf-GCN (full, $\alpha = 1$)	0.732±0.023	0.732±0.042	0.731±0.019	0.794±0.013	—

separate root transformation lets it fall back on a node's own features. Models that mix a node's representation with its description neighbors (GCN, GAT, Conf-GCN) lose 6.7–7.6 F1 points relative to ignoring the graph. The run-to-run standard deviation of F1 (0.016–0.019 for GCN-type models) is of the same order as the single-split differences in Table IV, which explains why a single split was misleading.

D. Ablations (RQ3)

Table VI isolates the confidence signal. The uniform-confidence control, which keeps the layer but discards confidence, is *at least as good* as the full model (F1 0.736 vs. 0.731, AUROC 0.795 vs. 0.794). Shuffled confidence, which preserves the marginal distribution but destroys node identity, and inverted weighting, which favors the *least* confident neighbors, both land within 0.5 F1 points of the full model and are not significantly different from it. Sharpening the weights does not help either: $\alpha = 2$ and $\alpha = 4$ slightly lower F1 (0.728, 0.727) and AUROC (0.793, 0.788). The behavior of Conf-GCN is therefore explained by its normalization, not by confidence.

The self-loop variants identify which part of the normalization matters. Adding a self-loop to the uniform layer recovers GCN-level performance (F1 0.742, AUROC 0.817 vs. GCN 0.740, 0.819); adding it to the confidence-weighted layer recovers the AUROC (0.815) but not the F1 (0.734). The 2.5-point AUROC gap between Conf-GCN and GCN in Table V is thus almost entirely due to the missing self-loop, i.e. to the node's own features being diluted by its neighbors. We answer RQ3 negatively: on this benchmark the confidence signal itself has no measurable benefit, and the precision-oriented behavior suggested by the single split is not a property of the method.

TABLE VII

TEST PRECISION OF EACH BASELINE AT A THRESHOLD TUNED ON ITS VALIDATION SET TO CONF-GCN'S VALIDATION RECALL (MEAN $\pm$ STD OVER 25 RUNS). Δ = CONF-GCN $-$ BASELINE; p: PAIRED WILCOXON, HOLM-CORRECTED.

Baseline	Rec. (base)	Prec. (base)	Prec. (Conf)	Δ	p
MLP (no graph)	0.733±0.038	0.850±0.021	0.732±0.023	-0.118	<0.001
GCN	0.730±0.038	0.740±0.022	0.732±0.023	-0.009	<0.001
GraphSAGE	0.731±0.035	0.848±0.022	0.732±0.023	-0.116	<0.001
GAT	0.731±0.039	0.743±0.022	0.732±0.023	-0.012	<0.001

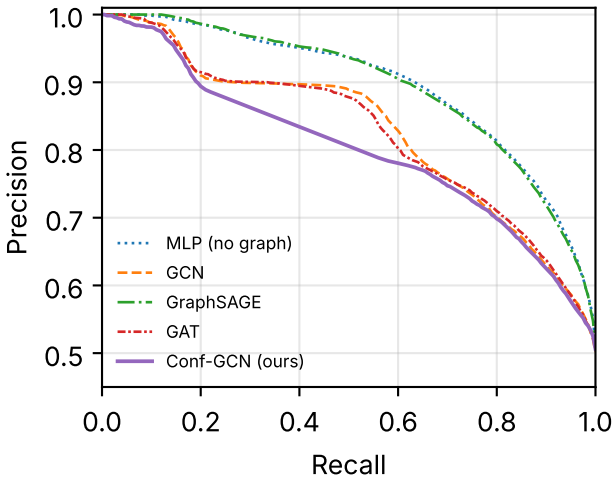


Fig. 2. Precision–recall curves on the test folds (precision interpolated on a common recall grid and averaged over the 25 runs). Description graph, $k = 10$.

E. Threshold-matched precision

Table VII asks whether a practitioner who wants Conf-GCN's operating point could obtain it more cheaply by moving the threshold of a baseline. At matched recall (0.730–0.733 for all models) GCN is 0.9 points and GAT 1.2 points *more* precise than Conf-GCN, and MLP and GraphSAGE are about 12 points more precise; all differences are consistent across runs ($p < 0.001$). The precision–recall curves (Fig. 2) show the same picture at every operating point: the Conf-GCN curve lies below GCN and GAT, which lie well below MLP and GraphSAGE. Any precision-oriented operating point is therefore better obtained by raising the threshold of a baseline.

F. Graph construction and density (RQ4)

The description graph is a weak structure: only 61.7% of its edges join accounts of the same class (50% would be random for a balanced sample), and the 30% of accounts with an empty description share an identical embedding, so their neighborhoods are essentially arbitrary among themselves. Table VIII shows that changing *only* the graph reverses the conclusion about GNNs. On the metadata graph (homophily 77.7%) every GNN beats the feature-only MLP: GraphSAGE reaches F1 0.840 and AUROC 0.921, GCN 0.826, Conf-GCN 0.824 and GAT 0.823, against 0.807 for the MLP (all $p < 2 \times 10^{-5}$). The feature graph, which mixes descriptions and metadata, is also clearly better than the description graph (every GNN gains 2–9 F1 points, $p < 10^{-6}$) but slightly worse than the metadata graph (GraphSAGE 0.825 vs. 0.840, $p < 10^{-6}$). GraphSAGE on the metadata graph is the best detector in this study: compared with the original pipeline (GCN on the description graph, F1 0.740) it gains 9.9 F1 points and 10.2 AUROC points, and it beats the MLP by 3.2 F1 points ($p < 10^{-6}$). Its separate root transformation lets it combine a node's own features with a neighborhood of similar accounts without diluting them, which is also why it was the only GNN that matched the MLP on the poor description graph.

Graph density matters much less than graph construction. On the description graph, GCN reaches F1 0.732/0.740/0.734 for $k = 5/10/20$ and Conf-GCN 0.729/0.731/0.726; GCN is ahead of Conf-GCN at every k (unadjusted $p = 0.15$, 0.002, < 0.001).

G. Confidence-aware extensions (RQ5)

Table IX evaluates the extensions of Section IV-E on the feature graph. All of them stay within 0.5 F1 points of their base model; none is significantly better, and AC-SAGE with self-training is slightly but significantly worse than GraphSAGE (-0.5 F1, $p = 0.018$). The adaptive design nonetheless fixes the main weakness of the original layer: on the description graph AC-GCN reaches F1 0.737 and AUROC 0.819 (GCN 0.740/0.819; Conf-GCN 0.731/0.794), i.e. the 2.5-point AUROC deficit of Conf-GCN disappears once confidence is applied on top of GCN's normalization instead of replacing it. The agreement-weighted GCN behaves the same (description graph F1 0.739, feature graph 0.818). On the metadata graph AC-GCN and AC-SAGE reach F1 0.825 and 0.835 (GCN 0.826, GraphSAGE 0.840), small but consistent losses (unadjusted $p = 0.042$ and 0.004).

The learned exponents explain this outcome. Starting from $\alpha = 1$, AC-GCN ends at $\alpha \approx (0.69, 0.69, 0.61)$ for its three layers and AC-SAGE at $(0.78, 0.77, 0.76)$: given the freedom to amplify confidence, the models *reduce* it, i.e. they move towards their baselines. Self-training is neutral when 60% of the nodes are labeled (feature graph: GCN 0.818 vs. 0.820, GraphSAGE 0.824 vs. 0.825; metadata graph: GraphSAGE 0.839 vs. 0.840), because the supervised signal is already abundant.

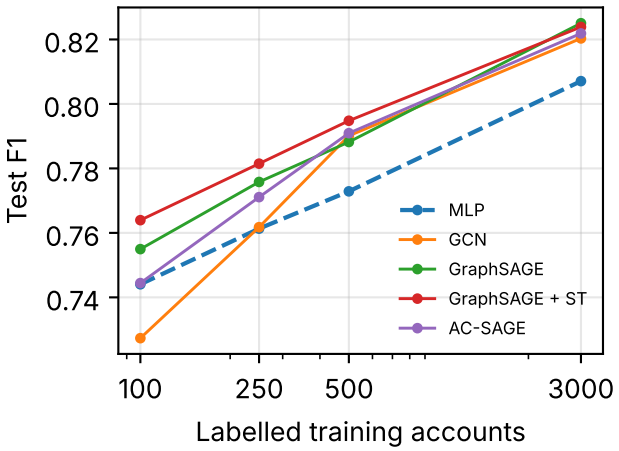


Fig. 3. Test F1 as a function of the number of labeled training accounts (feature graph).

H. Label-scarce regime (RQ6)

Labels are expensive in moderation, so Table X and Fig. 3 repeat the comparison with 100, 250 and 500 labeled accounts. GraphSAGE with confidence-based self-training is the best model at every budget from 100 to 500 labels: F1 0.764, 0.781 and 0.795 vs. 0.744, 0.761 and 0.773 for the MLP, a gain of 2.0–2.2 F1 points ($p = 0.002$ at 250 and at 500 labels, $p = 0.06$ at 100 labels; with 10 splits the smallest attainable two-sided Wilcoxon p is 0.002). Self-training adds to GraphSAGE where labels are scarce (+0.6 F1 at 250 labels, $p = 0.014$) and GCN also benefits (0.762 $\rightarrow$ 0.771 at 250 labels). Here confidence is useful, but as a filter for pseudo-labels rather than as a weight inside message passing: AC-SAGE is never significantly better than GraphSAGE ($p \geq 0.13$). The graph matters most when labels are few, because neighbors propagate information from labeled to unlabeled accounts.

I. Why the confidence signal has no effect

We trained Conf-GCN on the five folds of seed 0 and inspected the final confidence vector and the resulting edge weights. Averaged over the 25 main runs, the mean confidence on the real data rises only from 0.586 after the first epoch to 0.749 at epoch 100 (Fig. 4), in contrast to the synthetic graph where it saturates near 0.999; the 10th and 90th percentiles of the final confidences are 0.572 and 0.903. Inside a neighborhood the largest weight is a median 1.45$\times$ the smallest (95th percentile 1.82$\times$), and the weights deviate from uniform $1/\deg(i)$ by only 6.5% on average. The bounded-influence property of Section IV-C therefore binds in practice: the layer is very close to mean aggregation, which is why the uniform control reproduces it. The confidence signal is also only moderately informative: as a score for whether a test node is classified correctly it reaches an AUROC of only 0.66 (0.77 on the feature graph, where the weights still deviate from uniform by just 7.1%), and the softmax outputs of Conf-GCN are not better calibrated than those of the baselines (ECE

TABLE VIII

EFFECT OF GRAPH CONSTRUCTION ($k = 10$, MEAN $\pm$ STD OVER 25 RUNS). EDGE HOMOPHILY: DESCRIPTION GRAPH 0.617, METADATA GRAPH 0.777, FEATURE GRAPH 0.756.

	Description graph		Metadata graph		Feature graph	
Model	F1	AUROC	F1	AUROC	F1	AUROC
GCN	0.740±0.016	0.819±0.013	0.826±0.010	0.915±0.006	0.820±0.012	0.905±0.006
GraphSAGE	0.803±0.014	0.887±0.010	0.840±0.010	0.921±0.006	0.825±0.011	0.906±0.007
GAT	0.734±0.020	0.819±0.013	0.823±0.010	0.912±0.005	0.814±0.010	0.898±0.006
Conf-GCN (ours)	0.731±0.019	0.794±0.013	0.824±0.009	0.912±0.006	0.818±0.011	0.901±0.006
MLP (no graph)	F1 0.807±0.010, AUROC 0.890±0.008					

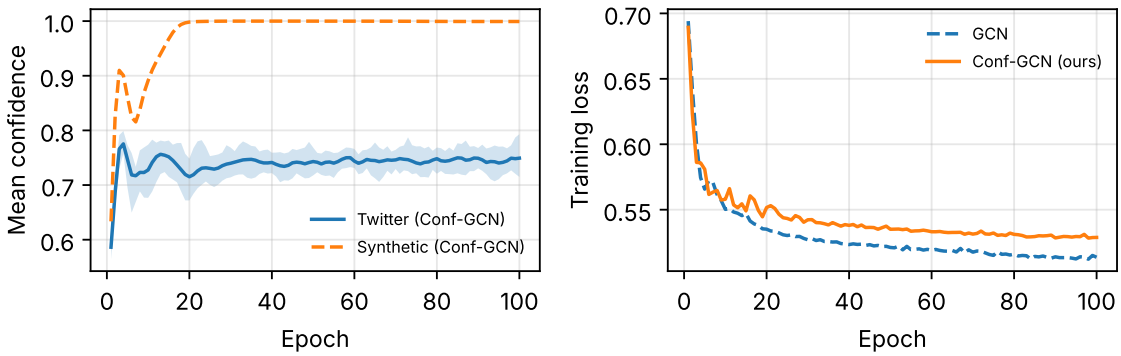


Fig. 4. Left: mean confidence of Conf-GCN over training (real data: mean and min–max range over 25 runs; synthetic: single run). Right: mean training loss of GCN and Conf-GCN on the real data.

TABLE IX

CONFIDENCE-AWARE EXTENSIONS ON THE FEATURE GRAPH (MEAN $\pm$ STD OVER 25 RUNS). p: PAIRED WILCOXON OF F1 VS. THE BASE MODEL OF EACH BLOCK, HOLM-CORRECTED WITHIN THE BLOCK.

Model	Prec.	Rec.	F1	AUROC	p (F1)
GCN	0.838±0.015	0.804±0.028	0.820±0.012	0.905±0.006	—
AC-GCN (learnable α)	0.832±0.016	0.805±0.025	0.818±0.011	0.904±0.006	0.519
Agreement-weighted GCN	0.831±0.016	0.806±0.024	0.818±0.011	0.903±0.007	0.519
GCN + self-training	0.837±0.016	0.800±0.030	0.818±0.012	0.904±0.006	0.519
AC-GCN + self-training	0.837±0.018	0.798±0.029	0.817±0.012	0.903±0.006	0.383
GraphSAGE	0.823±0.011	0.828±0.022	0.825±0.011	0.906±0.007	—
AC-SAGE (learnable α)	0.835±0.017	0.810±0.025	0.822±0.011	0.907±0.007	0.160
GraphSAGE + self-training	0.827±0.020	0.822±0.026	0.824±0.012	0.906±0.007	0.411
AC-SAGE + self-training	0.837±0.013	0.805±0.028	0.820±0.013	0.906±0.007	0.018

TABLE X

TEST F1 WITH FEW LABELED ACCOUNTS (FEATURE GRAPH; MEAN $\pm$ STD OVER 10 SPLITS; 3 000 LABELS = THE 25-RUN CV SETTING).

Model	100 labels	250 labels	500 labels	3 000 labels
MLP	0.744±0.015	0.761±0.012	0.773±0.010	0.807±0.010
GCN	0.727±0.023	0.762±0.016	0.790±0.011	0.820±0.012
GraphSAGE	0.755±0.018	0.776±0.012	0.788±0.010	0.825±0.011
AC-GCN	0.731±0.032	0.762±0.021	0.791±0.012	0.818±0.011
AC-SAGE	0.744±0.020	0.771±0.019	0.791±0.010	0.822±0.011
GCN + ST	0.733±0.023	0.771±0.012	0.792±0.012	0.818±0.012
GraphSAGE + ST	0.764±0.021	0.781±0.010	0.795±0.010	0.824±0.012

0.041 vs. 0.034 for GCN and 0.033 for MLP). Sharpening the weights with $\alpha > 1$ amplifies this partly wrong signal and lowers AUROC (Table VI). The cost of the mechanism is a second forward pass per epoch; on a shared two-core CPU a Conf-GCN run (100 epochs) took 16.9 s on average vs. 14.3 s for GCN.

VIII. DISCUSSION

A. Answers to the research questions

RQ1: the layers are correct: Conf-GCN converges on the synthetic graph and reduces exactly to mean aggregation for equal confidences; AC-GCN and AC-SAGE reduce exactly to GCN and GraphSAGE. **RQ2:** on the description graph Conf-GCN does not improve any metric over GCN, and all GCN-type models are beaten by an MLP. **RQ3:** the differences between Conf-GCN and GCN are explained by its normalization, not by confidence; a threshold-tuned GCN dominates it at matched recall. **RQ4:** graph construction dominates the aggregation rule; a metadata k-NN graph makes every GNN better than the MLP, and GraphSAGE on it is the best detector (F1 0.840, AUROC 0.921). **RQ5:** applying confidence on top of the baseline normalization removes Conf-GCN's AUROC loss, but neither learnable exponents nor agreement weights give a significant gain, and the learned exponents shrink below one. **RQ6:** with few labels, confidence is useful as a pseudo-label filter: GraphSAGE with self-training is the best model at 100–500 labels.

B. Practical significance

In moderation pipelines false positives are often more costly than false negatives, so a precision-oriented detector is attractive. Our threshold-matched analysis shows that Conf-GCN does not provide such an operating point more cheaply

than a baseline, so we withdraw the "implicit regularizer" interpretation suggested by the preliminary experiment. The practically useful outcome is different: building the similarity graph from account metadata and using an aggregator with a root connection raises precision *and* recall at the same time (GraphSAGE on the metadata graph: precision 0.842, recall 0.838 vs. 0.731/0.752 for the original GCN pipeline), at no extra labeling cost, and confidence-based self-training adds a further gain when only a few hundred accounts are labeled. Both changes are simple to deploy in an existing GNN pipeline.

C. Why reweighting is weak here

In the binary case $c_j \in [0.5, 1]$, so with $\alpha = 1$ two neighbors differ in weight by at most a factor of two, and on real data the weights deviate from uniform by only 6.5%. Larger exponents could break this bound, but the confidence of a neighbor is only moderately predictive of whether it is correct (AUROC 0.66–0.77), so amplifying it also amplifies errors; the learned exponents therefore decrease. Graph construction changes which accounts are neighbors at all, which is a much stronger intervention than rescaling their messages by at most a factor of two.

D. Absolute accuracy

The best model reaches about 84% accuracy on balanced data using profile metadata and description text only, without tweet history or an explicit follow graph. Reported accuracy for comparable feature sets varies widely across datasets and protocols [5], [11], so we rely on the in-paper comparisons.

E. Threats to validity

Graph construction: the k-NN graphs are derived from the same embeddings and metadata used as features, so edges encode similarity already present in the features; real follow or mention graphs may behave differently. **Labels:** partly derived from Botometer, which is itself a metadata-based detector; this may favor the metadata graph and metadata features. **Data:** we used an English-only cleaned redistribution and a 5 000-account subsample; the code also accepts the original release. **Statistics:** runs share data across folds and seeds, so Wilcoxon p-values are indicative. **Hyper-parameters:** the grid is small and was selected on seed 0; extensions reuse the settings of their base models, which favors neither side. **Confidence reliability:** softmax confidence is not guaranteed to be calibrated [14].

IX. Conclusion

We studied confidence-aware message passing for social bot detection, from a parameter-free Confidence-Aware GCN to an adaptive family with learnable exponents, agreement weights and confidence-based self-training, under a rigorous protocol with repeated cross-validation, paired tests, a threshold-matched comparison and ablations. The study yields a clear, positive design recipe: *build the similarity graph from account metadata and use an aggregator that keeps a node's own features*. With this recipe GraphSAGE reaches F1 0.840 and AUROC 0.921, 9.9 F1 points above the original GCN pipeline and 3.2 points above a strong feature-only MLP, and with only a few hundred labels confidence-based self-training on top of it gives the best detector. Confidence-weighted aggregation itself is correct, cheap and, in its adaptive form, as accurate as its baselines, but it does not improve them: its weights stay close to uniform and, when free to grow, the learned exponents shrink. The earlier single-split precision gain was sampling noise. We hope that the released code and per-run data help others to evaluate aggregation changes with repeated splits and to pay attention to how the graph is built.

X. Future Work

Three directions follow directly from these results. First, the confidence signal itself could be made more reliable, e.g. by temperature scaling or an ensemble estimate before it is used as a weight or a pseudo-label filter [14]. Second, graph construction could be learned rather than fixed, for example by selecting the metadata and text features that define neighborhoods on the validation set or by jointly learning the graph and the classifier. Third, evaluation on multi-relational graphs with explicit follow, mention, reply and retweet edges such as TwiBot-20 [9], with heterogeneous architectures such as BotRGCN [8], would show whether these conclusions transfer from similarity graphs to real social graphs.

Code and Data Availability

All code, configurations, per-run results (CSV), predicted probabilities and the scripts that generate every table and figure in this paper are available at https://github.com/solodancer235-star/confidence-aware-gcn-bot-detection. The full protocol, including the extensions, runs on a two-core CPU in about five hours.

Acknowledgment

The author thanks the School of Software Engineering at Astana IT University for guidance throughout the design and evaluation of this work.

References

[1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.

[3] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, 2017.

[4] C. A. Davis, O. Varol, E. Ferrara, A. Flammini, and F. Menczer, "BotOrNot: A system to evaluate social bots," in *Proc. 25th Int. Conf. Companion on World Wide Web*, 2016, pp. 273–274.

[5] K.-C. Yang, O. Varol, P.-M. Hui, and F. Menczer, "Scalable and generalizable social bot detection through data selection," in *Proc. AAAI*, vol. 34, no. 1, 2020, pp. 1096–1103.

[6] S. Cresci, "A decade of social bot detection," *Commun. ACM*, vol. 63, no. 10, pp. 72–83, 2020.

[7] S. Ali Alhosseini, R. Bin Tareaf, P. Najafi, and C. Meinel, "Detect me if you can: Spam bot detection using inductive representation learning," in *Companion Proc. World Wide Web Conf.*, 2019, pp. 148–153.

[8] S. Feng, H. Wan, N. Wang, and M. Luo, "BotRGCN: Twitter bot detection with relational graph convolutional networks," in *Proc. IEEE/ACM ASONAM*, 2021.

[9] S. Feng, H. Wan, N. Wang, J. Li, and M. Luo, "TwiBot-20: A comprehensive Twitter bot detection benchmark," in *Proc. 30th ACM CIKM*, 2021, pp. 4485–4494.

[10] S. Feng, Z. Tan, R. Li, and M. Luo, "Heterogeneity-aware Twitter bot detection with relational graph transformers," in *Proc. AAAI*, vol. 36, no. 4, 2022, pp. 3977–3985.

[11] D. Lee, S. Lee, and H. Yu, "Discriminative feature selection for multi-class Twitter bot detection," *IEEE Access*, vol. 9, pp. 32752–32762, 2021.

[12] S. Vashishth, P. Yadav, M. Bhandari, and P. Talukdar, "Confidence-based graph convolutional networks for semi-supervised learning," in *Proc. AISTATS*, 2019.

[13] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *ICML Workshop on Challenges in Representation Learning*, 2013.

[14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017, pp. 1321–1330.

[15] Y. Rong, W. Huang, T. Xu, and J. Huang, "DropEdge: Towards deep graph convolutional networks on node classification," in *Proc. ICLR*, 2020.

[16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, 2019, pp. 3982–3992.

[17] X. Zhu, Z. Ghahramani, and J. Lafferty, "Semi-supervised learning using Gaussian fields and harmonic functions," in *Proc. ICML*, 2003, pp. 912–919.

[18] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.

[19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[20] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[21] D. M. Gutiérrez, "Twitter human bots dataset," Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/davidmartngutirrez/twitter-bots-accounts

[22] Bot-Ani project, "Processed Twitter human/bot dataset (English accounts)," GitHub, 2024. [Online]. Available: https://github.com/Max-SF1/Bot-Ani-RWKV-twitter-bot-detection

[23] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," in *Proc. NeurIPS*, 2020.

[24] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. AAAI*, 2018, pp. 3538–3545.

[25] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.